Product Requirements Document

Digital Campus → Campus Services

Product: Campus Events & Clubs

Team: Team 12

Document status: Implementation-aligned PRD Last reconciled: 2026-11-10

> This document describes the current Campus Events & Clubs product boundary.
>
> The implementation contract in API_CONTRACT.md, the approved
>
> access workflow in ACCESS_WORKFLOW.md, and the root
>
> PROGRESS.md take precedence if this document becomes stale.
>
> Local test results are not evidence of full production acceptance.

---

1. Product overview

1.1 Product description

Campus Events & Clubs is a responsive web application for discovering campus activities, registering to attend, receiving a private ticket, and managing attendance. Organizers can create and publish activities, manage their own registrations, confirm attendance, plan event content, and request facilities support. Admins can review Organizer-access requests and view campus activity oversight information.

The core value remains:

> A Student can discover an activity, understand its details, register safely,
>
> retrieve their ticket, and manage their own registration.

1.2 Current implementation status

The current implementation uses Firebase Authentication, Cloud Firestore, a React/Vite frontend, and a persistent Express API. The frontend is hosted on Firebase Hosting and the API is hosted on Render. The repository contains automated backend, emulator, and browser checks; hosted cross-role acceptance and partner lifecycle acceptance remain separate verification activities.

The implementation is a documented deviation from the original Supabase/Vercel proposal. Lecturer approval, user interviews, and production CRUD acceptance must not be claimed unless separately recorded.

---

2. Scope

2.1 Implemented product scope

The product includes:

1. Google-only sign-in with server-side identity and token verification.
2. Student enrollment for verified `@lamduan.mfu.ac.th` accounts using a

separately supplied unique university ID.

3. Organizer access requests, policy-gated AI assessment, and Admin review.
4. Three application roles: Student, Organizer, and Admin.
5. Server-derived capabilities, including simultaneous Student and Organizer

workspaces for eligible enrolled Organizers.

6. Published and closed activity discovery with activity details.

7. Organizer-owned activity drafts, publishing, updating, and deletion of an unused draft.

8. Optional activity capacity, end time, and registration deadline.

9. Transactional registration, duplicate protection, capacity enforcement, and cancellation with history retention.

10. Private opaque QR/manual registration tickets.

11. Organizer-only ticket verification and explicit attendance check-in.

12. Organizer AI planning with reviewed agenda, checklist, and facilities suggestions, plus deterministic fallback.

13. Student-capability-protected Library learning-resource search on published or closed activities, with relevance gating and manual search.

14. Student-capability-protected, read-only Team 17 internship browsing.

15. Owner-reviewed Team 10 facilities requests and signed work-order updates.

16. Bounded self-service display-name editing for all provisioned roles.

17. Responsive, keyboard-accessible loading, empty, error, denied, and authoritative success states.

18. REST API, emulator-backed business-rule tests, and Playwright workflows.

2.2 Explicitly excluded

The product does not provide:

- Payments, paid tickets, or paid club memberships
- General club-membership administration
- Public chat, comments, likes, social feeds, or real-time messaging
- Native Android or iOS applications
- Student identity, roster, ticket, or check-in powers for Admins
- Public Admin appointment, arbitrary role-management, or event CRUD endpoints
- Student access for personal-email or unenrolled Organizers
- Borrowing or reservation of Library resources
- Internship applications inside this product
- Automatic facilities sending during save or publish
- External social-media publishing
- A fourth role such as Owner or Super Admin
- Multi-university enrollment

---

3. Users, roles, and capabilities

3.1 Student capability

An enrolled Student is a verified Google identity whose email is exactly in the `@lamduan.mfu.ac.th` domain and whose profile contains a unique, separately provided university ID. A Student can:

- Browse published and closed activities and view details.
- Register once for an available activity.
- View and cancel their own registrations, subject to check-in rules.
- Retrieve their own active ticket and use the Student workspace.
- Search Library learning resources for eligible activities.
- Browse the read-only Team 17 internship feed.
- Edit only their own display name.

The server, not the browser, derives this capability. An official university ID is never inferred from an email address.

3.2 Organizer capability

An approved or manually appointed Organizer can:

- Create, publish, update, and delete their own eligible drafts.
- Set optional schedules, capacity, deadlines, and private facilities notes.
- View registrations for activities they own.
- Verify tickets and explicitly check in active attendees for their activities.
- Use the planning assistant and request facilities support after review.
- Edit only their own display name.

An Organizer cannot manage another Organizer's activities or private roster. An enrolled MFU Student retains Student capability after Organizer approval and can switch between the two workspaces. Personal-email and unenrolled school Organizers do not gain Student capability.

3.3 Admin capability

Admin is a separate, trusted role. Admin can:

- View activity oversight information.
- List and inspect Organizer-access requests.
- Approve or reject an Organizer request with the required revision and evidence.
- Edit only their own display name.

Admin cannot access private Student rosters, tickets, check-in actions, Organizer activity CRUD, or public appointment/role-management endpoints. Admin appointment is a trusted operator operation.

3.4 Pending Organizer applicants

Any verified Google account may submit an Organizer request containing an organization or independent-organizer identity and a reason. Until approval or policy-based automatic approval, an applicant has access only to their own request/status page and has no campus-data access. Existing Student access is retained while a Student's request is pending.

The `CAMPUS_ORGANIZER_V1` policy may automatically approve any verified Google applicant, including personal-email applicants, only when all required criteria, supporting quotes, safety checks, identity rechecks, revision checks, and explicit server configuration pass. Uncertainty, provider failure, invalid output, changed identity, or a disabled policy leaves the request PENDING for Admin; AI never rejects a request or appoints an Admin.

---

4. Core journeys

4.1 Student discovery and registration

```
Google sign-in
  → complete Student enrollment when eligible
  → browse activities
  → open details
  → register
  → view authoritative confirmation
  → retrieve private ticket
```

4.2 Registration management

```
My registrations
  → inspect ACTIVE or CANCELLED history
  → cancel an active registration
  → refresh authoritative status
```

Cancellation is transactional and idempotent. A checked-in registration cannot be cancelled.

4.3 Organizer publishing and attendance

```
Organizer workspace
  → create draft
  → optionally use reviewed planning suggestions
  → save and publish
  → view owned roster
  → verify ticket
  → explicitly confirm check-in
```

Attendance is separate from registration status. Repeat scans are accepted as already checked in and do not increment the activity counter again.

4.4 Organizer access

```
Google sign-in
  → submit organization and reason
  → policy assessment or Admin review
  → approval creates/updates Organizer access atomically
```

Assessment input is limited to the submitted organization and reason. Tokens, unrelated identity data, and private rosters are not sent to the model.

---

5. Functional requirements

5.1 Identity and admission

• FR-01: Authenticate through Google Sign-In and verify the Firebase token,

provider, email verification, and revocation status server-side.

• FR-02: Derive the effective profile, role, and capabilities server-side;

reject client-supplied role, UID, email, university ID, or capabilities.

• FR-03: Create a Student profile only for an eligible school identity and a

unique, validated university ID.

• FR-04: Preserve existing Student identity, registrations, and capability

when the person is approved as an Organizer.

• FR-05: Make Organizer request submission, resubmission, AI assessment,

Admin review, and promotion atomic, concurrent-safe, revision-bound, and idempotent.

• FR-06: Allow a provisioned person to change only their own 1–80 character

display name through the bounded profile endpoint.

5.2 Activities

- FR-07: List published and closed activities with cursor pagination.
- FR-08: Show title, description, type, schedule, location, organizer,

availability, counts, optional limit, deadline, and end state.

- FR-09: Let an owning Organizer create a draft with required descriptive

fields and optional limits, deadline, end, and private facilities notes.

- FR-10: Publish only when required descriptive fields are valid.
- FR-11: Let the owner update or close an activity, but do not reopen a

closed activity or return a non-draft to draft.

- FR-12: Permit deletion only for an unused draft with no registrations.
- FR-13: Enforce that capacity edits cannot undercut active registrations.

5.3 Registration, tickets, and attendance

- FR-14: Permit a Student-capable person to register only for a published,

available activity.

- FR-15: Allocate seats, enforce deadlines/end times, and prevent duplicate

active registrations in one transaction.

- FR-16: Preserve cancelled registration history and free its seat.
- FR-17: Issue a private opaque ticket lazily and return it only to its

registration owner; never include ticket credentials in list DTOs, URLs, or logs.

- FR-18: Let only the owning Organizer verify and explicitly check in an

active attendee. Check-in must be atomic, repeat-safe, and separate from registration status.

5.4 Planning, learning, internships, and facilities

- FR-19: Provide Organizer-only planning suggestions for a timed agenda,

preparation checklist, and facilities requirements. Applying a suggestion requires section review and explicit acknowledgement of replacement.

- FR-20: Preserve manual text unless the user explicitly applies a reviewed

replacement. Save/publish and facilities Send remain separate.

- FR-21: Show Library learning resources only after a positive,

cached/coalesced relevance assessment for a published or closed activity. Manual Library search remains available; books and availability come from the Library provider, not the model.

- FR-22: Show read-only Team 17 internship listings only to Student-capable

accounts. Keep provider configuration independent of Maintenance and do not forward identity or credentials.

- FR-23: Allow only the owning Organizer to review and send a facilities

request. Save/publish does not send it; the server validates the authenticated partner receipt before atomically linking a work order.

- FR-24: Verify signed Maintenance webhooks over raw bytes, apply monotonic

lifecycle versions, preserve safe retries, and prevent status regression.

---

6. Non-functional requirements

• NFR-01 — Security: All protected operations require a verified Google

identity and server-side authorization. Browser Firestore access remains denied.

• NFR-02 — Privacy: Keep tokens, service-account keys, AI keys, partner

secrets, ticket credentials, and unnecessary personal data out of source, client bundles, logs, URLs, list projections, and committed evidence.

• NFR-03 — Integrity: Registration counts, active locks, capacity, deadlines,

check-in counters, admission decisions, and work-order links update atomically.

• NFR-04 — Availability: Provider failures must produce an explicit,

user-safe state. Planning may use a deterministic fallback; Library relevance failure hides AI suggestions but keeps manual search available.

• NFR-05 — Accessibility: Use responsive, keyboard-accessible interfaces

from 360 px through 1440 px, with focus return for controlled date/time dialogs and clear busy, empty, denied, error, and success states.

• NFR-06 — Maintainability: Keep the React/Vite frontend and persistent

Express API modular, preserve `.mjs` compatibility entrypoints, and do not move the worker into a serverless route.

• NFR-07 — Cost: The current release uses Firebase Hosting and an existing

Render service without enabling a paid plan. Render Free idle sleep is a known presentation limitation.

• NFR-08 — Verification: Automated checks use disposable emulators and

mocked AI/partner providers. They must not seed the connected project or consume real AI credits.

---

7. Business rules

7.1 Activity and registration state

Activity states are `DRAFT`, `PUBLISHED`, and `CLOSED`. Drafts are visible only to their owner; published and closed activities are visible to eligible authenticated users. Closed activities remain viewable and accept arrival check-in but reject new registrations. A closed activity cannot be reopened.

Registration states are `ACTIVE` and `CANCELLED`. The active registration lock is unique per Student-capable person and activity. Cancellation retains history, releases capacity, and is safe to repeat unless attendance has been recorded.

Capacity, registration deadline, and end time are optional for legacy compatibility. If configured, capacity is enforced in the registration transaction; deadline is no later than the start and end is later than the start.

7.2 Authorization

The backend checks role/capability and resource ownership on every protected operation. The client cannot select an effective role. Admin oversight does not grant Student, Organizer, roster, ticket, or check-in powers.

7.3 AI and partner boundaries

AI is advisory and bounded. It cannot establish verified affiliation, appoint Admin, reject Organizer requests, invent Library catalog facts, or silently overwrite user text. Partner calls are server-to-server only, explicitly triggered where required, authenticated, bounded, and retry-safe.

---

8. Data model and privacy

Cloud Firestore is the persistent data store. The main document families are:

- `users`: trusted profile, role, enrollment, and timestamps.
- `activities`: owner, public content, schedule, status, counts, optional limits, and private facilities linkage.
- `registrations`: Student/activity relation, status, timestamps, and safe attendance projection.
- `registrationTickets` and `ticketCodes`: private ticket credential records.
- Organizer request and AI-review records: submitted request revisions, bounded assessment evidence, decisions, and leases.
- Facilities, integration inbox/outbox, and work-order linkage records.

Relationships and authorization are enforced in the API and Firestore transactions/rules. No password is stored by this application; Google handles identity authentication. The application does not need to store home address, national ID, passport, financial information, grades, health data, location tracking, or unnecessary social data.

Participant projections expose only the minimum information required by an authorized owning Organizer. Ticket codes, QR values, tokens, private notes, provider credentials, and private audit internals are never exposed in list responses.

---

9. Architecture and technology

9.1 Runtime architecture

```
Browser
  ■■ React + TypeScript + Vite frontend
       ■■ HTTPS REST calls with Firebase ID token
            ■■ Persistent Node.js + Express API
                 ■■ Firebase Authentication verification
                 ■■ Cloud Firestore transactions/rules
                 ■■ Server-only AI provider
                 ■■ Server-only Team 10/17/Library integrations
```

The API is created by importing `backend/src/app.js`; that import must not initialize Firebase, listen on a port, or start the worker. Persistent runtime entrypoints retain the integration worker and `.mjs` compatibility wrappers.

9.2 Technology and deployment

Area	Current implementation
Frontend	React, TypeScript, Vite
Backend	Node.js, Express, Firebase Admin SDK
Identity	Firebase Authentication, Google provider
Database	Cloud Firestore
AI	Server-side AI/ML API with deterministic planning fallback
Hosting	Firebase Hosting frontend; existing Render backend
Verification	Node.js tests, Firebase emulators, Playwright
Storage	Firestore; no general user-upload storage

The original Supabase/PostgreSQL/Vercel design is historical and is not the current implementation. The current deployment URLs and release limitations are maintained in firebase-render.md.

---

10. API and interface contract

The complete request/response contract is maintained in API_CONTRACT.md. Representative routes include:

Method	Path	Purpose
GET	`/health`	Public process health and API marker
GET	`/api/me`	Current admitted profile and capabilities
POST/PATCH	`/api/me/profile`	Student enrollment or own-name edit
GET	`/api/activities`	Published/closed activity page
POST	`/api/activities`	Organizer creates an activity draft
PATCH/DELETE	`/api/activities/:id`	Owning Organizer activity lifecycle
POST	`/api/activities/:id/registrations`	Student registration
GET	`/api/me/registrations`	Own registration history
PATCH	`/api/registrations/:id/cancel`	Idempotent cancellation
POST	`/api/registrations/:id/ticket`	Own private ticket
POST	`/api/activities/:id/tickets/verify`	Owner Organizer ticket verification
POST	`/api/activities/:id/check-ins`	Owner Organizer check-in
GET/POST	`/api/me/organizer-request`	Own request submission/status
GET/PATCH	`/api/admin/organizer-requests`	Admin request review
POST	`/api/me/activity-plan`	Organizer planning suggestions
GET/POST	`/api/activities/:id/library-resources/*`	Student Library flows
GET	`/api/internships`	Student-capability internship browsing
POST	`/api/activities/:id/maintenance/requests`	Owner-reviewed facilities send
POST	`/webhooks/partner`	Signed Maintenance status webhook

Error responses use a consistent application error envelope. Authentication, authorization, validation, ownership, conflict, unavailable-provider, and internal failures must remain distinguishable and must not report success after an unsuccessful write.

---

11. Error handling and failure scenarios

Scenario	Required behavior
Invalid input or unknown fields	Reject before writing with a safe validation error
Missing/invalid/revoked identity	Return unauthorized; never trust the client
Wrong role/capability/ownership	Return forbidden or not found without leaking protected data
Duplicate or simultaneous registration	One transaction wins; the conflict is explicit
Full activity/deadline/end reached	Reject registration and preserve counts
Repeated cancellation/check-in	Return current authoritative state without double mutation
Provider timeout/failure	Show an explicit recovery state; use only approved deterministic fallback
Stale admission review or lease	Reject the stale result; preserve newer Admin/request state
Invalid webhook signature/version	Reject or acknowledge safely without changing state
Network failure after submission	Refresh authoritative state before offering a duplicate action

Logs and evidence must be sanitized. Do not log access tokens, raw tickets, service credentials, private partner payloads, or unrelated personal data.

---

12. Acceptance and verification

12.1 Required product checks

- [ ] A verified school Google account can complete Student enrollment with a unique university ID; a non-school account cannot enroll as Student.
- [ ] An enrolled Student can browse, register, retrieve a private ticket, and cancel before check-in.
- [ ] Capacity, deadline, duplicate registration, cancellation, and concurrent registration rules are enforced transactionally.
- [ ] The owning Organizer can create/publish an activity, view its roster, verify tickets, and confirm attendance; another Organizer cannot.
- [ ] An enrolled Student approved as Organizer retains Student capability and can switch workspaces.
- [ ] A personal-email applicant has only request/status access before approval.
- [ ] Admin can approve/reject Organizer requests but cannot use ticket, roster, check-in, or arbitrary event mutation endpoints.
- [ ] AI planning requires explicit review/application; AI failure has an honest fallback or recovery state.
- [ ] Irrelevant/uncertain Library relevance hides AI suggestions while manual search remains available.
- [ ] Internship browsing is Student-capability protected and read-only.
- [ ] Facilities are sent only after owner review, and webhook versions cannot regress an established status.
- [ ] Production configuration keeps server credentials out of the frontend.

12.2 Recorded verification boundary

The 2026-11-08 progress record reports a production frontend/API release, isolated emulator/browser verification, and user-reported school Google login. It also records known limits, including Render Free sleep and separate partner availability. These records do not prove full real-user, all-role, or live partner acceptance.

---

13. Constraints and risks

Constraints

- Team size and semester schedule are course constraints, not evidence of completed functionality.
- No paid plan, secret rotation, destructive schema change, or architecture migration without authorization.

• Do not seed the connected project or call real partner/AI services in automated tests.

• Preserve user uploads, existing compatibility entrypoints, stable partner contracts, leases, retries, and safe evidence.

Risks and mitigations

Risk	Mitigation
Incorrect authorization	Server-derived capabilities, ownership guards, emulator tests
Registration/check-in race	Firestore transactions and shared record locks
AI/provider uncertainty	Bounded calls, strict schema/evidence checks, human review, fallback
Partner outage or stale webhook	Stable retry identity, leases, raw-byte HMAC, monotonic versions
Render idle sleep	Treat as a known release limitation; verify service before demonstrations
Scope creep	Keep the core journey authoritative and document additions explicitly
Production evidence overclaim	Separate local, emulator, browser, and hosted results

---

14. Product boundary and future work

Current product boundary

```
Verified Google identity
  ■■ Enrolled Student capability
  ■    ■■ Discover → Register → Ticket → Cancel
  ■    ■■ Library learning resources
  ■    ■■ Team 17 internships
  ■■ Organizer capability
  ■    ■■ Draft → Publish → Roster → Check-in
  ■    ■■ Planning assistant
  ■    ■■ Reviewed facilities request
  ■■ Admin capability
       ■■ Organizer request review + read-only oversight
```

Future improvements

Potential future work includes notifications, calendar integration, richer club profiles and membership workflows, activity media, advanced analytics, mobile applications, broader partner contracts, and additional accessibility/local- ization work. These are not current requirements unless added to the approved implementation contract.

Success definition

The product succeeds when a Student can safely complete the discovery, registration, ticket, and cancellation journey; an Organizer can publish and operate their own activity; and Admin can review Organizer access without receiving unrelated powers or private Student data.